

Operating Systems — 89-231

Past Exam Questions Organized by Lecture Subject

Bar-Ilan University · Dept. of Computer Science · Prof. David Sarne et al.

This document collects questions from seven past finals (2022 Sem B Moed A & B, 2023 Sem B Moed A & B, and 2024 Sem B Moed A, B & C). Questions are grouped under the eight course subjects in lecture order, and within each subject they appear chronologically. When a question spans several subjects, it is filed under its dominant topic; secondary topics are noted in the details column. Provided answer keys were ignored — only the questions themselves were classified.

1. Von Neumann Architecture, Introduction & Basic OS Structure

Year & Semester	Question / Part	Details
2022, Sem B – Moed A	Q4 (a–e)	File-system control blocks, directory structures & IPC concepts — boot control block, FCB fields, single-level vs tree directories (overlaps file systems / IPC).
2022, Sem B – Moed A	Q5 (b–c)	OS process termination via <code>exit()</code> system call; CLI implementation methods (built-in vs external programs) and their trade-offs.
2024, Sem B – Moed C	Q5 a, b	<code>chmod</code> permission bits (765); “Command not found” and the shell <code>PATH</code> variable.

2. Processes

Year & Semester	Question / Part	Details
2022, Sem B – Moed A	Q5 (a)	Self-written mutual-exclusion code using an atomic <code>Power()</code> operation — correctness, syntax vs semantics, minimal fix (process synchronization).
2022, Sem B – Moed B	Q5 (a–b)	<code>fork()/kill()/signal</code> handlers; recursive fork tree, <code>getppid()</code> , zombie/orphan reasoning, output counting.
2023, Sem B – Moed A	Q1 (a–d)	Process state diagram (New/Ready/Running/Waiting/Terminated) — redundant transition, valid transitions, parent vs child state before <code>exec()</code> .
2023, Sem B – Moed A	Q5 (b–e)	<code>pipe()/fork()</code> producer-consumer, <code>SIGPIPE</code> , hard-link behaviour with <code>ln</code> , orphan PPID re-parenting, <code>chmod 777</code> .
2023, Sem B – Moed B	Q5 (a–e)	<code>kill()</code> system call, zombie process definition, pipe write/read after close, PPID after parent exit, client/server sockets.
2024, Sem B – Moed A	Q5 (a–c)	Signal handling & process control — <code>sigaction/pause/fork/execve</code> fill-in code; thread interleaving output count; zombie definition.
2024, Sem B – Moed B	Q5 (a–c)	Signal/process true-false statements; <code>sigaction/fork/execve/waitpid</code> fill-in code; concurrent thread output possibilities; zombie definition.
2024, Sem B – Moed C	Q3	Three concurrent processes sharing variable <code>x</code> — race conditions, possible final values, atomic load/store, which scheduling guarantees data consistency (overlaps scheduling).
2024, Sem B – Moed C	Q5 (c–e)	Semaphore busy-waiting, soft links across partitions, signals as async events, thread exit killing the process, zombie definition.

3. Threads

Year & Semester	Question / Part	Details
2023, Sem B – Moed B	Q1 (a–c)	Per-process vs per-user-thread uniqueness of resources (Data/Code/Files/Registers/Stack/Heap); mapping all user threads to one kernel thread — pros/cons & supported mapping models.
2024, Sem B – Moed B	Q5 (d)	Two threads sharing globals g1/g2 — number of distinct possible outputs (race conditions).
2024, Sem B – Moed C	Q5 (d)	Two threads sharing globals g1/g2 — number of distinct possible outputs (race conditions).

4. Scheduling

Year & Semester	Question / Part	Details
2022, Sem B – Moed A	Q1	Round-robin with quantum; event table (running process, ready-queue transitions) across blocks/terminates.
2022, Sem B – Moed B	Q4 (a)	Waiting time under Round Robin, SJF (preemptive) and Multi-Level Feedback Queue for I/O-bursting processes.
2023, Sem B – Moed A	Q2 (a–b)	FCFS & SJF (preemptive) — waiting/turnaround tables; effect of adding a 1-unit context-switch time.
2023, Sem B – Moed B	Q2 (a–b)	FCFS & SJF (preemptive, quantum 5, ctx-switch 1) — waiting/turnaround; effect of switching to non-preemptive.
2024, Sem B – Moed A	Q1 (a–c)	Preemptive SJF schedule of two I/O-bound processes; CPU utilization; whether another algorithm could do better; processor affinity.
2024, Sem B – Moed C	Q1 (a–d)	Algorithm giving min average waiting time (preemptive SJF); the four scheduler-invocation events & which are preemptive; processor affinity & cache; priority scheduling and starvation.

5. Deadlocks

Year & Semester	Question / Part	Details
2022, Sem B – Moed B	Q2 (a)	TestAndSet spin-lock with preemptive priority scheduling — can a deadlock occur? hold-and-wait debate (lock + CPU as two resources).
2022, Sem B – Moed B	Q4 (c)	Banker's algorithm — safe state & safe sequence; granting requests; max grantable to a thread without risking deadlock.
2023, Sem B – Moed A	Q4 (a–c)	Modified dining-philosophers with central chopsticks & a gate semaphore — deadlock-free values of i, worst-case concurrent eaters, starvation-free semaphore.
2023, Sem B – Moed B	Q4 (a–d)	Resource snapshot with Max-Need/Allocation — deadlock detection reasoning, Banker safety guarantee, granting specific requests.
2024, Sem B – Moed A	Q4	Resource-allocation graph (P1–P4, R1–R3) — inferring deadlock after edge/process/unit changes; reasons real OSes avoid Banker's algorithm.
2024, Sem B – Moed B	Q4 (a–b)	TestAndSet spin-lock with preemptive priority — deadlock possibility & hold-and-wait debate; readers-writers requirement not met by a single lock.
2024, Sem B – Moed C	Q4	Dining-philosophers monitor/semaphore solution — deadlock-freedom proof and starvation discussion.

6. Main Memory

Year & Semester	Question / Part	Details
2022, Sem B – Moed B	Q1	Paging without virtual memory + TLB — comparing two upgrades (halving memory access time vs doubling TLB hit-ratio) to minimize EAT.
2023, Sem B – Moed A	Q3 (a–b)	Paging address split (16-bit page #, 8-bit offset) — max process size; effect of various changes on user-space size (page size, 32-bit page #, etc.).
2024, Sem B – Moed B	Q2	Two-level vs three-level page table with TLB — solve for hit-ratio a giving equal EAT (feasibility check).
2024, Sem B – Moed C	Q2	Two-level vs three-level page table with TLB — solve for hit-ratio a giving equal EAT ($a > 1$ infeasible).

7. Virtual Memory

Year & Semester	Question / Part	Details
2022, Sem B – Moed A	Q2 (a–d)	Page-replacement page-fault counts (FIFO, Optimal, Counting, LRU); working-set window sizing; reasoning about thrashing & locality with Optimal.
2023, Sem B – Moed B	Q3 (a–b)	Replacing page tables with file-allocation schemes (contiguous/linked/indexed) — compute EAT; effect of using a FAT.
2024, Sem B – Moed A	Q3 (a–c)	Page-replacement page-fault counts — Optimal (3 frames), LRU (4 frames), FIFO (6 frames) on a reference string.
2024, Sem B – Moed B	Q2 a (true/false set)	Conditions under which LRU equals Optimal in page faults; working-set as minimum (not average); page-fault anomaly (Belady) with added RAM.

8. File System Interface & Implementation

Year & Semester	Question / Part	Details
2022, Sem B – Moed B	Q3 (a–b)	Free-block management — bit-vector vs linked-list trade-offs (contiguous need, FAT, near-full disk); I/O operations to append a block under Linked Allocation.